

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**ПАРАЛЕЛНА РАМКА ЗА МЕТАЕВРИСТИЧНА
ОПТИМИЗАЦИЯ НА КОМБИНАТОРНИ
ЗАДАЧИ: ПРОЕКТИРАНЕ, ПРОГРАМНА
РЕАЛИЗАЦИЯ И ЕКСПЕРИМЕНТАЛНА
ОЦЕНКА НА ПРОИЗВОДИТЕЛНОСТТА**

КУРСОВ ПРОЕКТ

по дисциплина „Метаевристика“

Разработил:

Алекс Цветанов, фак. № **[TODO: фак. номер]**

Специалност: Компютърно и софтуерно инженерство, ОКС „Магистър“

Преподавател:

Проф. д-р инж. Милена Лазарова, Доц. д-р Аделина Алексиева, Доц. д-р Георги Запрянов

София, 2027

СЪДЪРЖАНИЕ

УВОД	1
I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД	3
1.1. Комбинаторна оптимизация и мястото на метаевристиките	3
1.2. Метаевристики, включени в проекта	3
1.3. Паралелни схеми за изпълнение	4
1.4. Оценка на производителността	4
1.5. Еталонни набори от инстанции	5
1.6. Изводи от прегледа	5
II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА	6
2.1. Език и среда за изпълнение	6
2.2. Модел на паралелизъм	6
2.3. Представяне на задачата пред алгоритъма	7
2.4. Стратегия за обмен между нишките	7
2.5. Обобщение на избора	8
III. ПРОЕКТИРАНЕ НА СИСТЕМАТА	9
3.1. Слоеви и отговорности	9
3.2. Договор за задача	10
3.3. Договор за метаевристика	11
3.4. Схеми за паралелно изпълнение	12
3.5. Управление на състоянието и на случайността	13
3.6. Точки, в които проектът очаква измерване	13
IV. ПРОГРАМНА РЕАЛИЗАЦИЯ	14
4.1. Организация на изходния текст и изграждане	14
4.2. Генератор на псевдослучайни числа	15
4.3. Инкрементално оценяване и кеширано състояние	15
4.4. Избор на съседен ход при задачата на търговския пътник	15
4.5. Откъде идват разстоянията	16
4.6. Паралелно изпълнение и синхронизация	16

4.7. Векторизация на оценяването	17
4.8. Тестване	17
V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ОЦЕНКА	19
5.1. Изследвани въпроси	19
5.2. Апаратна и програмна среда	19
5.3. Еталонни инстанции	20
5.4. Критерий за спиране и изчислителен бюджет	21
5.5. Брой повторения и обобщаване	22
5.6. Измервани величини	23
5.7. Профилиране	23
5.8. Заплахи за валидността	23
VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ	25
6.1. Проверка на условията за валидност	25
6.2. Цена на едно оценяване на целевата функция	25
6.3. Как се избира съседният ход	26
6.4. Качество на четирите метаевристики при равен бюджет	28
6.5. Сравнение между паралелните схеми	29
6.6. Ускорение и ефективност спрямо броя нишки	30
6.7. Фалшиво споделяне	33
6.8. Разпределение на времето в една итерация	33
6.9. Обобщение на резултатите	34
ЗАКЛЮЧЕНИЕ	36
ЛИТЕРАТУРА	38
ПРИЛОЖЕНИЯ	39

УВОД

Голяма част от практически важните задачи за комбинаторна оптимизация са изчислително трудни, така че точното им решаване е приложимо само за малки входове. Вместо точно решение, инженерната практика използва *метаевристики*: общи стратегии за търсене, които не гарантират оптимум, но намират добро допустимо решение в приемливо време. Систематичният преглед в [1] показва, че тези стратегии се различават не по задачата, която решават, а по начина, по който балансират *изследване* на пространството от решения и *задълбочаване* в перспективен негов участък.

Точно тази обща форма прави метаевристичните подходящи за паралелно изпълнение: търсенето се състои от много слабо свързани опити, а не от една дълга зависима верига от изчисления. Класификацията на паралелните схеми в [2] обаче показва, че лесното паралелизиране е подвеждащо. Различните схеми, независимо многократен старт, споделяне на общо най-добро решение и островен модел с обмен между популации, се различават не само по постигнатото ускорение, а и по това дали изобщо решават същата задача: кооперативните схеми променят траекторията на търсенето, тоест променят и качеството на решението, а не само времето за получаването му.

Оттук следва и проблемът, около който е организиран настоящият курсов проект. Твърдение от вида „реализацията е паралелна и затова е по-бърза“ не е проверимо, докато не бъдат фиксирани едновременно три неща: наборът от еталонни входни инстанции, критерият за спиране и начинът, по който се сравнява качеството на получените решения. Без тези три условия измереното ускорение може да идва от по-слаба работа на нишка, а не от по-добро използване на машината.

Целта на проекта е да се проектира и реализира паралелна рамка за метаевристична оптимизация на комбинаторни задачи, в която няколко метаевристики и няколко схеми за паралелно изпълнение стоят зад общ договор, и да се направи количествена оценка на производителността и на качеството на решенията при контролирани условия.

За постигането на целта са формулирани следните **задачи**:

1. преглед на теорията на метаевристичните и на схемите за паралелното им изпълнение;
2. анализ на възможните проектни решения, тоест език и модел на паралелизъм, начин на представяне на задачата и стратегия за обмен на информация между нишките, и обосновка на направения избор;

3. проектиране на архитектура, в която задачата, метавристиката и схемата за паралелно изпълнение са три независими точки на разширение;
4. програмна реализация на избраните метавристики и схеми, с внимание към цената на синхронизацията и към разположението на данните в паметта;
5. изграждане на методика за измерване, която фиксира входните инстанции, критерия за спиране, броя повторения и начина на обобщаване на резултатите;
6. експериментално измерване на ускорението, на ефективността и на качеството на решенията, профилиране на реализацията и сравнителен анализ на алтернативите.

Обхватът на работата е рамката, метавристиките в нея и експерименталната оценка. Извън обхвата остават разпределеното изпълнение върху няколко машини, изпълнението върху графични ускорители и точните методи за решаване, които се използват само като източник на известни оптимални стойности за еталонните инстанции.

Структура на изложението. Раздел I разглежда предметната област и литературата. Раздел II излага разгледаните алтернативи и обосновава избора между тях. Раздел III представя проектирането, а Раздел IV програмната реализация. Раздел V описва методиката за измерване, която е предпоставка за валидността на всичко след нея. Раздел VI представя получените резултати и анализа им, а заключението обобщава изводите и очертава посоките за по-нататъшна работа.

I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД

Разделът е подреден от общото към частното: първо задачата, която метаверистиките решават, после отделните семейства метаверистики и техните първоизточници, и накрая схемите за паралелно изпълнение, които са същинският предмет на проекта.

1.1. Комбинаторна оптимизация и мястото на метаверистиките

Задачата за комбинаторна оптимизация се задава с крайно множество от допустими решения и с целева функция върху него. Множеството е крайно, но нараства толкова бързо с размера на входа, че изчерпателното му обхождане е неприложимо още при скромни размери. Точните методи заобикалят това чрез отсичане на цели подмножества, но при трудните задачи и те се изправят пред същия ръст.

Метаверистиките приемат друг компромис: отказват се от гаранцията за оптималност и в замяна получават контрол върху изразходваното време. Общото между тях е описано в [1] като стратегия за управление на подчинена евристика, която сама по себе си би заседнала в локален оптимум. Подробното систематично изложение на семействата, на техните компоненти и на въпросите по реализацията им е дадено в [3].

1.2. Метаверистики, включени в проекта

1.2.1. Симулирано закаляване

Симулираното закаляване приема и влошаващи ходове с вероятност, която намалява с понижаването на управляващ параметър, наречен температура по аналогия с металургичния процес. Методът е формулиран в [4], където е и връзката с алгоритъма на Метрополис. Практическото поведение на метода зависи изцяло от схемата на охлаждане и от дефиницията на съседството, което го прави удобна отправна точка за измервания: една и съща реализация дава различно качество при различна схема, и разликата е измерима.

1.2.2. Търсене с забрани

Търсенето с забрани, въведено в [5], използва краткосрочна памет за скорошните ходове и забранява връщането към тях за определен брой стъпки. Така търсенето се

измъква от локален оптимум, без да разчита на случайност. Забраненият списък е структура с честа проверка за принадлежност, тоест точката, в която представянето на данните в паметта пряко влияе върху времето за една итерация.

1.2.3. Генетичен алгоритъм

Популационните методи, чиято основа е положена в [6], поддържат множество решения едновременно и ги комбинират чрез оператори за кръстосване и мутация. За настоящия проект тези методи са важни с това, че оценката на пригодността на отделните индивиди е независима, тоест естествено се разпределя между нишки.

1.2.4. Мравчена колония

Оптимизацията с мравчена колония [7] изгражда решения стъпка по стъпка, като използва обща за колонията матрица от следи, обновявана след всяка итерация. Тази обща матрица е и основната разлика спрямо предходните три метода: тук споделеното състояние не е оптимизация, а част от самия алгоритъм, което прави метода естествен тест за цената на синхронизацията.

1.3. Паралелни схеми за изпълнение

Прегледът в [2] и сборникът [8] класифицират паралелните метаевристики по това какво точно се разпределя: отделни независими търсения, оценката на съседството в рамките на едно търсене, или популация, разделена на подпопулации с периодичен обмен. Трите варианта имат различни свойства. Независимият многократен старт не изисква комуникация и мащабира почти линейно, но не използва информацията, натрупана от останалите нишки. Кооперативните схеми използват тази информация, но плащат за нея със синхронизация и с риск от преждевременно сходимост към едно и също решение. Островният модел стои между двете и въвежда честотата на обмена като допълнителен управляващ параметър.

1.4. Оценка на производителността

Сравнението на паралелни реализации изисква модел, който отделя ограничението от изчисленията от ограничението от паметта. Моделът на покривната линия в [9] свързва постигнатата производителност с аритметичната интензивност на изчислението и дава

горна граница, спрямо която измерената стойност се тълкува. За метаевристиките този модел е приложим главно към оценяването на целевата функция, което е и най-често изпълняваната част от кода.

1.5. Еталонни набори от инстанции

Сравнимостта с публикувани резултати изисква публично достъпни инстанции с известни или най-добри известни стойности. За задачата на търговския пътник такъв набор е TSPLIB [10], а за редица други комбинаторни задачи, включително задачата за раницата и задачите за покриване на множество, такъв набор е OR-Library [11]. Конкретните инстанции, използвани в проекта, са изброени в Раздел V.

1.6. Изводи от прегледа

Прегледаните източници описват метаевристиките и паралелните схеми поотделно и всеки от тях фиксира по едно сравнение: класификацията в [2] подрежда схемите за паралелизъм, а обзорът в [1] подрежда стратегиите за търсене. Липсва общата рамка, в която трите оси, задача, стратегия и схема за изпълнение, се менят независимо и се измерват в едни и същи условия. Именно това е празнината, която проектът адресира, и тя е практическа, а не теоретична: без такава рамка всяко сравнение между две публикувани реализации е и сравнение между два различни набора инстанции, два различни критерия за спиране и две различни настройки на параметрите. Проектът не предлага нова метаевристика и не твърди подобрене спрямо публикувани резултати; той предлага условия, при които сравнението между вече известни методи е проверимо, и показва с измерване докъде тези условия стигат.

II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА

Проектът има четири независими проектни решения: език и среда за изпълнение, модел на паралелизъм, начин на представяне на задачата пред алгоритъма и стратегия за обмен на информация между нишките. Разделът разглежда алтернативите по всяко от тях и обосновава направения избор. Обосновката е предварителна там, където зависи от измерване, и в такива случаи е отбелязана изрично.

2.1. Език и среда за изпълнение

Разгледани са три възможности. Език с управлявана среда за изпълнение би съкратил времето за разработка, но въвежда събиране на боклука в най-горещия цикъл на търсенето и прави измерването на времето за една итерация зависимо от фактор извън контрола на програмата. Език с явно управление на паметта и без среда за изпълнение дава пълен контрол, но за част от нужните конструкции няма зряла стандартна поддръжка. Избран е C++ по стандарта ISO/IEC 14882:2020 [12], защото дава едновременно контрол върху разположението на данните в паметта и стандартни средства за паралелизъм, тоест позволява измерването да отразява алгоритъма, а не средата под него.

2.2. Модел на паралелизъм

Първата възможност е директно управление на нишки. Тя дава най-точен контрол върху свързването на нишка с ядро и върху момента на синхронизация, но прехвърля цялата координация върху програмиста. Втората е задачно ориентирана среда с крадене на работа, която се справя по-добре с неравномерно натоварване, но добавя слой, чиято цена трябва да бъде отчетена отделно при измерването. Третата е директивен подход над цикли, който е най-икономичен откъм код, но се вписва зле в търсене с нерегулярна структура.

Избран е първият вариант, с явно управление на нишките чрез `std::jthread` и с атомарни операции за споделеното състояние, защото предметът на измерването е именно цената на синхронизацията и тя не бива да бъде скрита зад чужд планировчик.

Задачно ориентираната среда не влиза като втори, сравнителен вариант. Причината е, че сравнението между двата модела би било валидно само ако и двете реализации са

еднакво добре настроени, а настройването на втората отнема усилие, което в рамките на проекта е по-полезно вложено в измерването на трите схеми за обмен. Тя остава в предложенията за по-нататъшна работа.

2.3. Представяне на задачата пред алгоритъма

Възможните решения са две. Динамичен полиморфизъм чрез виртуални функции дава един общ договор и позволява алгоритъмът да бъде компилиран веднъж, но вкарва непряко извикване в най-често изпълнявания път, тоест в оценяването на целевата функция. Статичен полиморфизъм чрез шаблони и концепции премахва това извикване и отваря целевата функция за вграждане и за векторизация, но увеличава времето за компилация и размера на кода.

Първоначалното намерение беше статичният вариант за горещия път и динамичен договор само на нивото на конфигуриране. Реализиран е динамичният вариант навсякъде, и причината е измерване, което към момента на планирането още не беше налично.

Профилът в §6.8 показва, че оценяването на целевата функция заема между 8,7 и 10,3 на сто от една итерация, докато генерирането на съседен ход заема между 70 и 73. Тоест непрякото извикване, което статичният вариант премахва, стои пред около една десета от времето, а не пред основната му част, и премахването му не може да върне повече от няколко процента. Срещу това стои цена, която не е хипотетична: с шаблони всяка от четирите метаевристики се компилира отделно за всяка от трите задачи, тоест дванадесет отделни инстанции, а изборът на алгоритъм и на задача по низ от командния ред, от който измервателната програма зависи, изисква отделна диспечерна таблица. Един абстрактен клас с виртуални функции постига същото с една.

Изборът следователно е динамичен полиморфизъм навсякъде, обоснован с измерване, а не с очакване. Уговорката е, че той е валиден за тези три задачи: при задача, чието оценяване е скъпо, а генерирането на ход евтино, съотношението се обръща и изборът трябва да бъде преразгледан.

2.4. Стратегия за обмен между нишките

Трите разгледани схеми следват класификацията от [2]: независим многократен старт, кооперативна схема със споделено най-добро решение и островен модел с периодична миграция. Те не са взаимно изключващи се, а образуват редица с нарастваща

комуникация, затова и трите са реализирани и влизат в сравнението. Твърдението, че по-голямата комуникация се изплаща с по-добро качество, е хипотеза за проверка, а не изходно допускане.

2.5. Обобщение на избора

Четири решения и основанието за всяко от тях са събрани в Табл. 1 (Табл. 1). Основание, което зависи от измерване, сочи раздела, в който измерването се намира.

Таблица 1. Обобщение на проектните решения

Решение	Разгледани алтернативи	Избрано	Основание
Език и среда	управлявана среда; език без среда за изпълнение; C++20	C++20	Контрол върху паметта и стандартни средства за паралелизъм, без събирач на боклук в горещия цикъл.
Модел на паралелизъм	явни нишки; задачно ориентирана среда; директиви над цикли	явни нишки	Предметът на измерването е цената на синхронизацията и тя не бива да е скрита зад чужд планировчик.
Представяне на задачата	статичен полиморфизъм с шаблони; динамичен полиморфизъм	динамичен	Оценяването е под 11 на сто от итерацията, §6.8, тоест премахнатото непряко извикване не си струва дванадесетте инстанции.
Обмен между нишките	независим старт; споделено най-добро; островен модел	и трите	Образуват редица с нарастваща комуникация; кое се изплаща е хипотеза за проверка, а не допускане, и е проверена в §6.5.

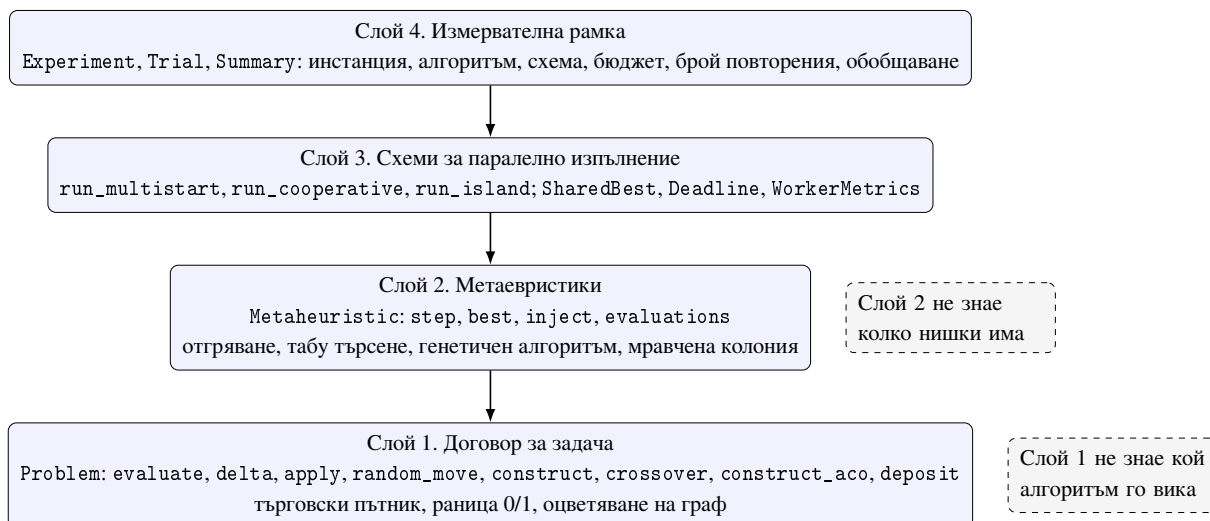
III. ПРОЕКТИРАНЕ НА СИСТЕМАТА

Проектирането изхожда от едно изискване: трите неща, които се менят в експериментите, тоест задачата, метаевристиката и схемата за паралелно изпълнение, трябва да се менят независимо едно от друго. Ако смяната на схемата за изпълнение налага промяна в кода на алгоритъма, всяко сравнение между схемите става сравнение и между две различни реализации на алгоритъма, тоест престава да бъде измерване на това, което се твърди.

3.1. Слоеви и отговорности

Системата е разделена на четири слоя. Най-долният описва задачата и предоставя представяне на решение, целева функция и оператор за преминаване към съседно решение. Над него стои слойът на метаевристиките, който работи само през този договор и не знае коя конкретна задача решава. Третият слой съдържа схемите за паралелно изпълнение, които управляват няколко екземпляра на търсенето и обмена между тях. Най-горният слой е конфигурирането и измерването: избор на инстанция, на алгоритъм, на схема и на критерий за спиране, и записване на резултатите в машинно четим вид.

Посоката на зависимостите е само надолу и е показана на Фиг. 1 (Фиг. 1). Слойът на задачите не знае нищо за метаевристиките, слойът на метаевристиките не знае нищо за нишките, а слойът на схемите не знае коя задача се решава. Практическата последица е, че добавянето на четвърта задача не изисква промяна в нито един от четирите алгоритъма, а добавянето на пета схема за изпълнение не изисква промяна в нито една задача.



Фигура 1. Слоеве и посока на зависимостите

3.2. Договор за задача

Договорът за задача е минималният набор от операции, които всяка метаевристика изисква. Той включва тип на решението, оценяване на решението, генериране на начално решение и обхождане на съседството. Отделно се предвижда операция за *инкрементално оценяване*, тоест изчисляване на промяната в целевата функция при конкретен ход, без пълно преизчисляване. Тази операция не е удобство, а проектно решение с пряко следствие върху производителността: при задачата на търговския пътник пълното оценяване е с линейна сложност спрямо броя на върховете, а промяната при размяна на две ребра се изчислява с константен брой операции. Измереното отношение между двете е дадено в Раздел VI.

Окончателният списък от операции е фиксиран след реализацията и на трите задачи, а не след първата, за да не бъде обобщен от един частен случай. Операциите се разделят на четири групи:

- **Оценяване.** `evaluate` за пълно пресмятане, `delta` за промяната при ход, `apply` за прилагане на хода заедно с обновяване на кешираното състояние, `feasible` за проверка на допустимост.
- **Генериране.** `construct` за конструктивна евристика и `random_solution` за случайно допустимо решение. Конструктивната евристика е рандомизирана: детерминистична такава би дала на всички нишки едно и също начално решение и би обезсмислила независимия многократен старт.

- **Съседство.** `random_move` и `move_key`, като вторият връща атрибута на хода, използван от табу списъка.
- **Специфични за алгоритъм.** `crossover` за генетичния алгоритъм и `pheromone_size`, `construct_aco` и `deposit` за мравчената колония. Тези операции принадлежат на задачата, а не на алгоритъма, защото равномерна рекомбинация би разрушила пермутация, а рекомбинация с подредба е безсмислена за двоичен вектор.

3.2.1. Единно представяне и кеширано състояние

И трите задачи използват едно и също представяне, вектор от цели числа. При задачата на търговския пътник това е пермутация на градовете, при раницата е индикаторен вектор, а при оцветяването е цвят за всеки връх. Единното представяне прави хода, рекомбинацията и миграцията една и съща операция върху паметта навсякъде и позволява договорът да бъде обикновен абстрактен клас без шаблони.

Само решението обаче не е достатъчно. Инкременталното оценяване при раницата изисква текущото общо тегло, а генерирането на ход при търговския пътник изисква обратната пермутация, тоест позицията на всеки град в маршрута. И двете са част от състоянието на кандидата, а не на задачата, защото задачата се използва едновременно от няколко нишки и трябва да остане неизменяема. Затова кандидатът е тройка от решение, скалар и целочислен масив, а тяхното значение се определя от задачата (Табл. 2).

Таблица 2. Кеширано състояние на кандидата по задачи

Задача	Решение	Скалар	Масив
Търговски пътник	пермутация	не се ползва	обратна пермутация
Раница 0/1	индикатор 0/1	общо тегло	не се ползва
Оцветяване	цвет на връх	не се ползва	не се ползва

3.3. Договор за метавевристика

Метавевристката се разглежда като обект със състояние, който изпълнява стъпки, а не като функция, която връща решение. Причината е в паралелните схеми: за да може една нишка да получи решение отвън и да продължи от него, търсенето трябва да бъде прекъсваемо в определени точки, а състоянието му да бъде видимо. Договорът включва изпълнение на една стъпка, четене на текущото най-добро решение, приемане на решение

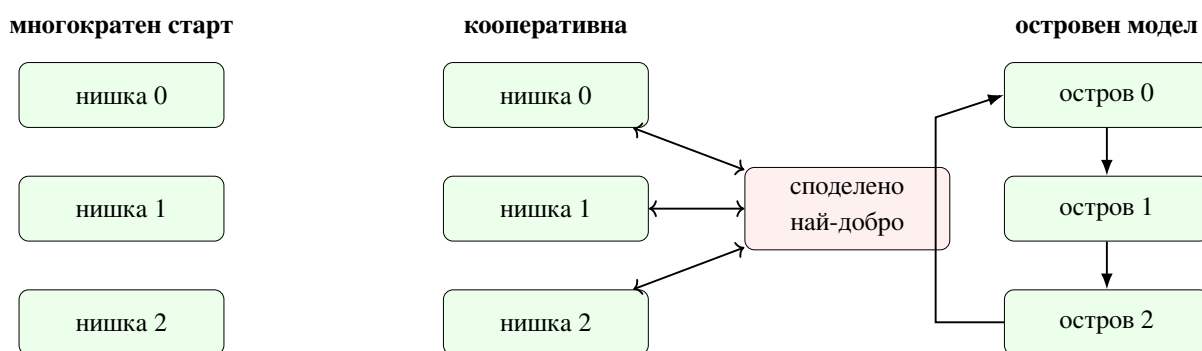
отвън и брояч на извършените оценявания.

Една стъпка означава различно нещо при различните алгоритми: един ход при отгряването, едно поколение при генетичния алгоритъм и едно преминаване на цялата колония при мравчения алгоритъм. Затова бюджетът никога не се задава в стъпки, а в изтекло време или в брой оценявания на целевата функция. По същата причина периодът на обмен между нишките се брои в оценявания, а не в итерации: период, разумен за отгряването, което изпълнява милиони итерации в секунда, означава нула обмени за мравчената колония, която изпълнява около сто и петдесет.

3.4. Схеми за паралелно изпълнение

И трите схеми използват един и същ договор за метаевристика и се различават само по това какво се обменя и кога (Фиг. 2).

- **Независим многократен старт.** Всяка нишка изпълнява самостоятелно търсене със собствен генератор на псевдослучайни числа. Обмен няма, а резултатът е най-доброто от всички решения. Тази схема е и еталонът, спрямо който се измерва цената на комуникацията в останалите две.
- **Кооперативна схема със споделено най-добро решение.** Нишките четат и обновяват обща най-добра стойност. Обновяването е рядко, а четенето е често, тоест схемата се проектира около това съотношение: цената се чете от атомарна променлива без заключване, а самото решение се защитава с мютекс.
- **Островен модел.** Нишките са разделени на подпопулации, които обменят решения през пощенски кутии по пръстен. Топологията е пръстен, а не пълен граф, защото при пълен граф едно добро решение се разпространява за един обмен и схемата се изразжда в кооперативната.



Фигура 2. Трите схеми за паралелно изпълнение и обменът между работниците

3.5. Управление на състоянието и на случайността

Възпроизводимостта е изискване към проекта, а не пожелание. Всяка нишка получава собствен генератор на псевдослучайни числа с извлечено от общо начално число подсемя, така че едно изпълнение да може да бъде повторено точно при същия брой нишки. Изпълнение с различен брой нишки не е и не може да бъде побитово еднакво, затова сравненията между конфигурации се правят по разпределението на резултатите от няколко независими изпълнения, а не по единично изпълнение.

Възпроизводимостта е точна само при бюджет, зададен в брой оценявания: всяка нишка получава равен дял и спира на него, независимо от скоростта на машината. При бюджет, зададен във време, броят итерации зависи от натоварването и резултатът не е побитово повторим. Това е свойство на измерването, а не дефект, и е записано като такова.

3.6. Точки, в които проектът очаква измерване

Измервателните точки са фиксирани заедно с методиката от Раздел V и се събират в структура за всяка нишка поотделно (Табл. 3). Структурата е подравнена на 64 байта, тоест на кеш линия, а дали това подравняване има значение е измерено, а не предположено; резултатът е в §6.7.

Таблица 3. Величини, записвани за всяка нишка

Величина	Какво отговаря
iterations	колко стъпки е направил работникът
evaluations	колко оценявания на целевата функция, единната мярка за работа
improvements	колко пъти работникът е подобрил собственото си най-добро
offers	колко пъти е публикувал решение към споделено състояние
adopted	колко пъти е приел решение отвън
cas_failures	неуспешни атомарни обновявания, тоест мярка за състезание
sync_wait_ns	време, прекарано вътре в споделеното най-добро или в кутия

IV. ПРОГРАМНА РЕАЛИЗАЦИЯ

Разделът описва как проектът от Раздел III е сведен до работеща програма и кои решения при реализацията имат пряко отражение върху измереното време. Няколко от тях бяха взети погрешно на първия опит и поправени след измерване; те са описани такива, каквито се случиха, защото грешката и измерването, което я откри, са част от резултата.

4.1. Организация на изходния текст и изграждане

Изходният текст е разделен на заглавни файлове с договорите и с частите, които трябва да бъдат вградими в мястото на употреба, и на файлове с реализация за останалото (Табл. 4). Изграждането е с CMake.

Таблица 4. Модули на реализацията

Файл	Съдържание
anneal/rng.hpp	генератор xoshiro256++ със засяване splitmix64
anneal/core.hpp	Solution, Move, Candidate, договорите Problem и Metaheuristic
anneal/problems.hpp	трите задачи и генераторите на инстанции
anneal/algorithms.hpp	конфигурации на четирите метаверистики, правилото на Метрополис, табу таблицата
anneal/parallel.hpp	Budget, SharedBest, Deadline, трите схеми
anneal/experiment.hpp	измервателната рамка
anneal/stats.hpp	медиана, квартили, ускорение, ефективност, относително отклонение
src/tsp.cpp и др.	реализации на трите задачи
src/algorithms.cpp	четирите метаверистики
src/parallel.cpp	трите схеми и споделеното състояние
apps/demo.cpp	демонстрация с една команда
benchmarks/bench.cpp	измервателната програма
tests/check.hpp и др.	изпълнител на тестове и 28 теста

Проектът няма нито една задължителна външна зависимост. Това не е предпочитание към стил, а условие за възпроизводимост: изграждане, което изисква мениджър на пакети, е изграждане, което трето лице няма да повтори. Затова изпълнителят на тестове е деветдесет реда в tests/check.hpp вместо външна рамка, а измерването на време използва std::chrono::steady_clock вместо външна библиотека за измервания.

4.2. Генератор на псевдослучайни числа

Използван е `xoshiro256++` със засяване чрез `splitmix64`, а не `std::mt19937_64`. Причината не е скорост, а възпроизводимост: стандартните разпределения като `std::uniform_int_distribution` не са специфицирани до бит, тоест едно и също начално число дава различни числа при друга стандартна библиотека. Тегленето в интервал също е написано изрично, с отхвърляне, защото `nxt() % n` е отклонено към малките стойности, когато n не дели 2^{64} .

4.3. Инкрементално оценяване и кеширано състояние

Инкременталното оценяване е отделна операция в договора. За задачата на търговския пътник промяната при размяна на две ребра засяга четири разстояния и се изчислява без обхождане на маршрута. За раницата промяната засяга едно или две тегла, но изисква текущото общо тегло, за да се провери допустимостта; затова кандидатът носи това тегло като кеширан скалар и то се обновява при всяко прилагане на ход. Без този кеш инкременталното оценяване при раницата би било линейно и би обезсмислило самото си съществуване.

Предпоставката за коректност е една: кешираното състояние трябва да бъде обновявано от `apply` при всеки ход и възстановявано от нула при всяко създаване на кандидат. Това е и точно твърдението, което тестът `knapsack_delta_matches_full_evaluation` проверява: след всеки от петстотин случайни хода кешираната стойност и стойността, пресметната отново, съвпадат. Грешно инкрементално оценяване не се проявява като срыв и не изглежда грешно, а просто съобщава цена, каквато решението няма.

4.4. Избор на съседен ход при задачата на търговския пътник

Това е решението с най-голямо измерено влияние в целия проект и първата му версия беше грешна.

Първата реализация избираше две позиции равномерно и обръщаше отсечката между тях. При инстанция с хиляда града нито отгряването, нито табу търсенето успяваха да подобрят маршрута от конструктивната евристика, с която тръгват. Причината се вижда от разпределението на предложените ходове: 98 на сто от тях влошават маршрута,

а медианата на влошаването е 355 при дължина на маршрута около 29 000. Началната температура се калибрира именно от това разпределение, тоест тя излиза огромна и търсенето се превръща в случайно блуждаене между маршрути, по-лоши от началния.

Реализацията сега избира ход по друг начин: тегли град, тегли един от шестнадесетте му най-близки съседа и намира позицията на този съсед през обратната пермутация, която кандидатът носи. Така въведеното ребро винаги е късо. Ефектът е измерен, а не предположен: делът на подобряващите предложения се вдига от 0,17 на 3,3 на сто, тоест двадесет пъти, а медианата на влошаването пада от 355 на 82.

Отделно от това е ограничението върху дължината на обръщаната отсечка. Първоначално тя беше ограничена до петдесет позиции с мотива, че така една итерация остава с константна сложност. Измерването показва обратното: при два милиона предложения низходящо търсене стига до 27 579 с ограничение и до 24 965 без него. Ограничението забранява точно далечните поправки, от които маршрутът от най-близък съсед има нужда. Пълните числа са в §6.3.

4.5. Откъде идват разстоянията

Очевидната оптимизация при задачата на търговския пътник е предварително пресметнатата матрица на разстоянията, която спестява корен квадратен при всяко оценяване. Тя е правилна само докато се събира в кеша. Матрицата е n^2 числа с плаваща запетая, а координатите са $2n$; при четири хиляди града матрицата е 128 MB и всяко оценяване плаща обръщение към паметта. Реализацията избира между двете по размер, като прагът е измерен, а не избран по документация; числата са в §6.2.

4.6. Паралелно изпълнение и синхронизация

Нишките се управляват с `std::jthread`, чието времетраене е обвързано с обхвата, така че изход по изключение да не оставя работещи нишки. Всяка нишка притежава собствен обект на търсенето, създаден вътре в самата нишка от фабрика, тоест единственото споделено състояние в процеса е това, което схемата въвежда нарочно.

Споделеното най-добро решение е двойка от атомарна цена и защитено с мютекс решение. Атомарната цена е бързият път: всяка нишка я чете често и я пише рядко, така че мютекс върху четенето би сериализирал честия случай заради редкия. Обновяването е по схемата „сравни и размени“ в цикъл, а неуспешните опити се броят и се докладват като

мярка за състезание. Самото решение не може да бъде атомарно, затова се записва под мютекс и условието се проверява повторно вътре в критичната секция: само атомарната операция би позволила нишка, спечелила размяната, да бъде изпреварена, преди да е копирала решението, и публикуваната цена да описва друго решение.

Периодът на обмен се брои в оценявания на целевата функция, а не в итерации. Първата версия броеше итерации и резултатът беше, че мравченият алгоритъм не обменяше нищо: при период от 128 итерации и общо около четиридесет итерации в рамките на бюджета условието не се изпълняваше нито веднъж. Измерените стойности на `adopted` в Раздел VI са нула точно за тази конфигурация, преди поправката.

Отделните нишки работят върху свои данни, подравнени на 64 байта, така че две нишки да не пишат в една и съща кеш линия. Ефектът е измерен отделно в §6.7.

Спирането по време не се проверява на всяка итерация. Едно четене на часовника струва приблизително колкото една итерация на отгряването, тоест проверка на всяка итерация би измервала главно часовника. Класът `Deadline` увеличава разстоянието между проверките, докато две последователни проверки се разделят от поне една милисекунда, което ограничава закъснението след срока до около една милисекунда.

4.7. Векторизация на оценяването

От трите целеви функции само една има формата, която компилатор може да векторизира: целевата функция на раницата е умножение и натрупване върху два съседни масива без разклонение и без непряка адресация. Затова тя е написана точно така, а очевидната форма с разклонение е запазена като втора реализация, за да бъде разликата измерена, а не спорена. Дължината на маршрута е сума с непряка адресация през пермутация и не се векторизира в нито една от двете си форми. Резултатът, включително случаите, в които разликата е в полза на формата с разклонение, е в §6.2.

4.8. Тестване

Тестовите са 28 и покриват четири групи твърдения: че инкременталното оценяване дава същия резултат като пълното за произволни ходове и при трите задачи, че всяко върнато решение е допустимо, че при фиксирано начално число и бюджет в оценявания резултатът е побитово възпроизводим, и че инстанциите с обявен оптимум наистина го имат. Последната група е по-важна, отколкото изглежда: тестът за оцветяването построява

засадената подялба и проверява, че тя дава нула конфликта, а тестът за решетката построява маршрут с дължина точно n пъти стъпката. Без тези свидетели обявеният оптимум е само твърдение, а всяко относително отклонение спрямо него би било невярно.

Тестовете за производителност са отделени от тестовете за коректност, защото първите са чувствителни към натоварването на машината, а вторите не бива да бъдат.

V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ОЦЕНКА

Методиката фиксира предварително какво се измерва, върху какво, при какви условия и как се обобщава. Целта е измерванията да са сравними помежду си и разликите да могат да бъдат отдадени на конкретна причина, а не на разлика в условията.

5.1. Изследвани въпроси

Експериментите отговарят на пет въпроса, всеки от които е формулиран така, че да може да бъде опроверган:

1. Как се променя времето до фиксиран критерий за спиране с нарастване на броя нишки и как се отнася полученото ускорение към идеалното?
2. Дава ли кооперативната схема по-добро качество на решението от независимия многократен старт при равен изчислителен бюджет, и каква е цената на обмена?
3. Каква част от времето отива за оценяване на целевата функция, каква за генериране на съседни и каква за синхронизация?
4. Как се подреждат четирите метаевристики по качество на решението при еднакъв бюджет и върху едни и същи инстанции, и зависи ли подредбата от задачата?
5. Помага ли векторизацията на целевата функция там, където формата ѝ я допуска?

5.2. Апаратна и програмна среда

Всички измервания са направени на една машина, чиято конфигурация е записана в Табл. 5 (Табл. 5). Стойностите са прочетени от самата машина, а не от документация.

Таблица 5. Конфигурация на машината, на която са направени измерванията

Компонент	Стойност
Процесор	AMD Ryzen 5 3600, 6 физически ядра, 12 логически
Обявена честота	3950 MHz
Операционна система	Windows, версия 10.0.26200
Компилятор	g++ 15.2.0, MinGW-w64, x86_64-posix-seh
Стандарт	C++20, -std=c++20, без разширения
Флагове	-O3 -DNDEBUG, тоест конфигурация Release на CMake
Без -march=native	по подразбиране изключено, за преносимост
Система за изграждане	CMake 4.3.2, генератор Ninja 1.13.2
Часовник	std::chrono::steady_clock
Външни зависимости	няма

Динамичното управление на честотата не е изключвано, защото средата не позволява това да бъде направено надеждно. Това е ограничение, което се записва тук, а не се премълчава: то увеличава разсейването между повторенията и е една от причините резултатите да се представят с междуквартилен размах, а не само с медиана. Машината не е изпълнявала друго обявено натоварване по време на измерванията.

5.3. Еталонни инстанции

Проектът не изтегля инстанции. Причината е практическа: изграждане или измерване, което изисква мрежов достъп, не се повтаря от трето лице години по-късно. Вместо това инстанциите се генерират, а обявената стойност на всяка от тях е или доказана в затворен вид, или пресметната точно (Табл. 6). Реализиран е и четец на формата TSPLIB [10] за EUC_2D, за да може публикувана инстанция да бъде подадена отвън, но нито едно от измерванията по-долу не зависи от изтеглен файл.

Таблица 6. Използвани инстанции и произход на обявената стойност

Инстанция	Размер	Стойност	Откъде идва стойността
uniform-1000	1000 града	неизвестна	Случайни точки. Оптимумът не е известен и класът връща NaN, тоест качеството се докладва като дължина, а не като отклонение.
grid-24x24	576 града	5760	Решетка със стъпка 10. Никое ребро не е по-късо от стъпката, тоест n пъти стъпката е долна граница; за четна страна съществува маршрут, който я достига, и тестът го построява.
knapsack-400	400 предмета	15771	Точен оптимум, пресметнат с динамично оптимизиране върху генерираната инстанция. Проверен срещу изчерпателно изброяване при 16 предмета.
coloring-150-k8	150 върха, 8 цвята	0 конфликта	Графът е генериран от засадена правилна подялба, тоест оцветяване с нула конфликта съществува по построение и тестът го предявява.

Инстанцията с точки върху окръжност също е реализирана и тествана, но не се използва за сравнения. Причината е измерена: върху точки в изпъкнало положение конструктивната евристика „най-близък съсед“ връща направо оптималния маршрут, тоест инстанцията не разграничава нищо.

5.4. Критерий за спиране и изчислителен бюджет

Използват се два режима, като всеки резултат посочва в кой от двата е получен.

- **Фиксирано време.** Измерва се качеството, постигнато за еднакъв бюджет от стенно време. Този режим сравнява алгоритми и схеми помежду им.
- **Фиксирано качество.** Измерва се времето до достигане на предварително зададена стойност. Този режим и само той отговаря на въпроса за ускорението: при фиксиран бюджет от време повече нишки просто извършват повече работа, което не е ускорение, а по-голяма машина.

Прагът за втория режим не се избира на око. Изпълнява се пилотна серия от пет изпълнения с една нишка и бюджет 400 ms, и за праг се взема *най-лошото* постигнато качество. Така една нишка достига прага при всяко повторение, тоест базовата линия на ускорението е измерена, а не е изтекъл срок. Праг, който една нишка не достига, би дал безкрайно ускорение и не би значел нищо.

Бюджет може да бъде зададен и в брой оценявания на целевата функция. Той се използва в тестовите, защото е единственият режим, при който резултатът е побитово възпроизводим: всяка нишка получава равен дял и спира на него. Броят оценявания се докладва и в режима с фиксирано време, но с уговорката, че едно инкрементално оценяване и едно пълно оценяване се броят еднакво, а не струват еднакво; затова основната мярка за бюджет е времето, а броят оценявания е съпътстваща величина.

5.5. Брой повторения и обобщаване

Метаевристиките са стохастични, тоест единично изпълнение не е резултат. Всяка конфигурация се изпълнява 11 пъти с различни начални числа. Броят е избран след пробно изпълнение: при пет повторения междуквartilният размах на времето до праг беше съизмерим с медианата, тоест пет не различаваха съседни конфигурации. Началните числа са $\text{base_seed} + i$, така че отделно повторение може да бъде възпроизведено, без да се повтарят предходните.

Резултатът се представя с медиана и с междуквartilен размах, защото разпределението на времената е силно изкривено надясно и средната стойност се влияе от единични бавни изпълнения. Квartilите се пресмятат с линейна интерполация между съседните рангове, тоест определението, използвано по подразбиране от NumPy и от R. Определението се посочва изрично, защото трите разпространени определения се разминават при малки извадки, а извадките тук са малки.

Относителното отклонение се дефинира като $(\text{намерено} - \text{известно}) / \max(1, |\text{известно}|)$. Знаменателят е ограничен отдолу, защото при оцветяването известната стойност е точно нула, където относително отклонение не е дефинирано; в този случай величината се изразжда в абсолютна разлика и се докладва като брой конфликти, а не като процент.

5.6. Измервани величини

Записват се време на изпълнение, брой оценявания, стойност на най-доброто намерено решение и отклонение от известната стойност. От тях се изчисляват ускорението, тоест отношението на медианното време при една нишка към медианното време при p нишки, и ефективността, тоест ускорението, разделено на p . И двете се изчисляват от медиани, а не от двойка единични изпълнения. Отделно се записват величините за всяка нишка от Табл. 3: време в синхронизация, брой неуспешни атомарни обновявания, брой публикувани и брой приети решения.

5.7. Профилиране

На тази машина не беше изпълнен профайлър и такъв не се твърди. Наличният на Linux `perf` го няма, а инструментирание с четене на часовника около всяка операция би струвало повече от самата операция: едно четене на `steady_clock` е съизмеримо с една итерация на отгряването.

Използваният метод е *аблация*. Измерва се цикъл, съдържащ само първата част от работата на една итерация, после цикъл с първите две части, и така нататък, като разликата между два реда е цената на добавената част. Методът има известни ограничения и те се посочват заедно с резултата: той не отчита взаимодействие между частите, тоест приписва на всяка част това, което тя добавя *при вече наличните преди нея*, и не вижда преразпределение на изпълнението между конвейерите на процесора. За грубо разпределение на времето по участъци е достатъчен, а за твърдения за кешови пропуски и инструкции на такт не е, затова такива твърдения не се правят и моделът на покривната линия [9] не се прилага.

5.8. Заплахи за валидността

Методиката признава пет известни ограничения. Първо, резултатите са получени на една машина и не се пренасят автоматично върху друга архитектура. Второ, качеството на метасвристика зависи силно от настройката на параметрите ѝ, тоест сравнението между алгоритми е сравнение между конкретни настройки, а не между методите изобщо. Трето, при фиксиран бюджет схемите с комуникация извършват по-малко оценявания за същото време, което трябва да бъде отчетено при тълкуването. Четвърто, динамичното управление

на честотата не е изключено. Пето, инстанциите са генерирани, тоест те имат структура, която публикуваните набори нямат; заключенията се отнасят за измереното, а не за класа задачи изобщо.

VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ

Разделът представя резултатите, получени по методиката от Раздел V. Всяка таблица посочва командата, която я е произвела, така че отделна клетка да може да бъде проследена до изпълнение. Всички числа са измерени на машината от Табл. 5.

6.1. Проверка на условията за валидност

Възпроизводимостта при фиксирано начално число е проверена автоматично, а не твърдяна. Тестът `multistart_matches_best_of_workers` изпълнява една и съща конфигурация два пъти с бюджет от 400 000 оценявания и четири нишки и изисква побитово съвпадение както на цената, така и на върнатото решение; тестът минава. Тестът `metaheuristics_are_reproducible` изисква същото за всеки от четирите алгоритъма при еднакво начално число и различен резултат при различно начално число.

При бюджет, зададен във време, побитова възпроизводимост няма и не се твърди. Мярката за разсейване в този режим е междуквартилният размах, който присъства във всяка таблица. Динамичното управление на честотата не е изключено, което е записано в §5.2 като известно ограничение.

6.2. Цена на едно оценяване на целевата функция

Първата измерена величина е цената на самото оценяване, защото тя определя мащаба на всичко останало. Резултатите са в Табл. 7 и Табл. 8 (Табл. 7).

Таблица 7. Целева функция на раницата: с и без разклонение. Команда: `anneal_bench objective`

Форма	n	ns на оценяване	ns на елемент	отношение
без разклонение	256	187,3	0,732	1,00
с разклонение	256	218,4	0,853	1,17
без разклонение	1024	775,4	0,757	1,00
с разклонение	1024	1053,4	1,029	1,36
без разклонение	4096	3131,8	0,765	1,00
с разклонение	4096	4749,4	1,160	1,52

Целевата функция на раницата е умножение и натрупване върху два съседни масива без разклонение и без непряка адресация, тоест единствената от трите, която

компиляторът може да векторизира. Формата без разклонение е между 1,17 и 1,52 пъти по-бърза от очевидната форма с разклонение, като предимството расте с размера: при 256 предмета масивите се събират в L1 и разликата е малка, а при 4096 предмета формата без разклонение задържа 0,765 ns на елемент, докато формата с разклонение се влошава до 1,160. Това е положителен резултат, но с ограничен обхват: спечеленото е около 1,6 ns на предмет, а както §6.8 показва, пълното оценяване изобщо не е на горещия път.

Таблица 8. Дължина на маршрута: през матрица на разстоянията и от координати.
Команда: `anneal_bench objective`

Източник	n	ns на оценяване	ns на град	отношение
матрица	200	150,2	0,751	1,00
координати	200	460,4	2,302	3,07
матрица	1000	788,6	0,789	1,00
координати	1000	2275,8	2,276	2,89
матрица	2000	2135,5	1,068	1,00
координати	2000	4541,8	2,271	2,13
матрица	4000	31768,5	7,942	1,00
координати	4000	9156,5	2,289	0,29

Дължината на маршрута е сума с непряка адресация през пермутация и не се векторизира в нито една от двете форми. Тук измерването опроверга очакването. Предварително пресметнатата матрица спестява корен квадратен и е по-бърза, докато се събира в кеша: при 1000 града тя дава 0,789 ns на град срещу 2,276 за координатите. При 4000 града матрицата е 128 MB, стойността ѝ скача на 7,942 ns на град и координатната форма става 3,5 пъти по-бърза. Формата от координати струва постоянните 2,3 ns на град при всички размери, защото два масива от n числа остават в кеша при всяко n , разгледано тук.

Заклучението е, че „предварително пресметни, за да спестиш корена“ е вярно само под условие за размер, и реализацията избира по този праг вместо да го предполага. Тъй като времето за корен квадратен е постоянно, а за търсене в матрицата расте, тази крива е и единственото място в проекта, където поведението на паметта се вижда пряко.

6.3. Как се избира съседният ход

Това е измерването с най-голям ефект в целия проект (Табл. 9). Всички редове тръгват от един и същ маршрут, получен от конструктивната евристика, и изпълняват два

милиона предложения при чисто низходящо търсене, тоест приемат само подобряващи ходове. Колоната е качеството, което самият генератор на ходове може да достигне, без метаевристика над него.

Таблица 9. Генератор на ходове при два милиона предложения, инстанция uniform-1000.
Команда: anneal_bench neighbourhood

Генератор	Обхват	Дължина	Подобрения	ns на ход
начало, най-близък съсед	—	29307,2	—	—
равномерен 2-opt	999	25086,8	3396	52,1
равномерен 2-opt	50	27443,9	12857	53,9
списък съседи	50	27579,4	63330	83,5
списък съседи	200	26874,9	62803	79,4
списък съседи	999	24964,5	66761	58,3

Четат се два независими ефекта. Първият е обхватът на обръщането. Ограничаването му до петдесет позиции влошава крайния резултат от 24964,5 на 27579,4, тоест с 10,5 на сто, защото забранява точно далечните поправки, от които маршрутът от най-близък съсед има нужда. Първоначалната реализация имаше точно това ограничение, въведено с мотива, че задържа една итерация с константна сложност; мотивът беше верен, а следствието нежелано.

Вторият ефект е списъкът от най-близки съседи. Той вдига дела на подобряващите предложения от 0,17 на 3,3 на сто, тоест двадесет пъти, но при два милиона предложения крайният резултат се различава малко: 24964,5 срещу 25086,8. На пръв поглед оттук следва, че списъкът не си заслужава. Не следва, и причината е в друго измерване: списъкът свива разпределението на влошаващите ходове, чиято медиана пада от 355 на 82. Симулираното отгряване калибрира началната си температура точно от това разпределение, тоест при равномерно теглене веригата тръгва около четири пъти по-гореща от нужното. В тази конфигурация нито отгряването, нито табу търсенето успяваха да подобрят собствения си начален маршрут в рамките на бюджета: при седем начални числа и бюджет 250 ms и двата алгоритма съобщаваха точно 28528,4 като най-добро, най-добро и най-лошо между 25177 и 29445, тоест стойността на конструктивната евристика.

Оттук следва и общо наблюдение, което е по-важно от конкретните числа: качеството на метаевристика в рамките на фиксиран бюджет се определя от съседството поне толкова, колкото от самата стратегия за търсене. Сравнение между метаевристики,

направено върху лошо избрано съседство, измерва съседството.

6.4. Качество на четирите метаевристики при равен бюджет

Табл. 10 дава качеството при 250 ms на повторение, 11 начални числа и една нишка (Табл. 10).

Таблица 10. Качество при бюджет 250 ms, една нишка, 11 повторения. Команда:
anneal_bench quality

Инстанция	Алгоритъм	Медиана	МКР	Отклонение	Оценявания
uniform-1000	отгряване	25481,11	141,36	—	1 941 903
uniform-1000	табу	24488,29	181,21	—	4 341 681
uniform-1000	генетичен	27011,09	342,06	—	52 058
uniform-1000	мравчен	27336,82	197,26	—	1249
grid-24x24	отгряване	5851,13	29,71	1,582 %	2 404 751
grid-24x24	табу	5760,00	8,28	0,000 %	4 280 241
grid-24x24	генетичен	5760,00	10,32	0,000 %	84 400
grid-24x24	мравчен	5760,00	21,54	0,000 %	3617
knapsack-400	отгряване	15724,00	7,00	0,298 %	3 875 215
knapsack-400	табу	15527,00	26,50	1,547 %	5 078 961
knapsack-400	генетичен	15753,00	19,00	0,114 %	105 956
knapsack-400	мравчен	15707,00	10,00	0,406 %	145
coloring-150-k8	отгряване	0,00	0,00	0 конфликта	1 196 431
coloring-150-k8	табу	0,00	0,00	0 конфликта	747 441
coloring-150-k8	генетичен	4,00	2,00	4 конфликта	157 642
coloring-150-k8	мравчен	11,00	4,50	11 конфликта	2081

Подредбата зависи от задачата и това е основното наблюдение. Табу търсенето е най-добро при двете инстанции на търговския пътник и най-лошо при раницата, където изостава с 1,55 на сто от точния оптимум, докато генетичният алгоритъм е на 0,114 на сто. Отгряването и табу търсенето решават оцветяването точно, а популационните методи не. Твърдение от вида „метод X е по-добър от метод Y“ не се поддържа от тези измервания; поддържа се само твърдение, обвързано със задачата.

Броят оценявания се различава с четири порядъка между отгряването и мравчения алгоритъм. Това не е дефект в мравчения алгоритъм, а разлика в цената на една итерация: едно преминаване на колонията строи шестнадесет пълни решения. Именно затова бюджетът се задава във време, а не в стъпки, и затова периодът на обмен между нишките

се брой в оценявания.

Инстанцията grid-24x24 се решава точно от три от четирите алгоритъма и следователно не ги разграничава. Тя е запазена в набора, защото е единствената инстанция на търговския пътник с известен оптимум и служи за проверка, че отклонението изобщо се пресмята правилно, а не за подреждане.

6.5. Сравнение между паралелните схеми

Табл. 11 сравнява трите схеми при еднакъв бюджет от стенно време, осем нишки и табу търсене (Табл. 11).

Таблица 11. Трите схеми при 8 нишки, 300 ms, 11 повторения, табу търсене. Команда:
anneal_bench schemes

Инстанция	Схема	Медиана	МКР	Синхр. ms	CAS	Приети
uniform-1000	многократен	24320,92	116,99	0,000	0	0
uniform-1000	кооперативна	24095,71	137,22	23,372	8	1614
uniform-1000	островна	24136,65	144,82	12,733	0	808
knapsack-400	многократен	15552,00	20,00	0,000	0	0
knapsack-400	кооперативна	15591,00	53,00	0,253	0	54
knapsack-400	островна	15608,00	24,50	15,024	0	47
coloring-150-k8	многократен	0,00	0,00	0,000	0	0
coloring-150-k8	кооперативна	0,00	0,00	0,045	0	38
coloring-150-k8	островна	0,00	0,00	2,184	0	33

Комуникацията се изплаща, но малко, и цената ѝ е измерима. При задачата на търговския пътник кооперативната схема подобрява медианата с 0,93 на сто спрямо независимия многократен старт, а островната с 0,76 на сто. При раницата подредбата се обръща и двете схеми с обмен изпреварват независимата, съответно с 0,25 и 0,36 на сто. При оцветяването и трите схеми решават инстанцията точно и не се разграничават.

Цената на комуникацията е 23,4 ms сумарно за всичките осем работника при бюджет от 300 ms всеки, тоест около 1 на сто от общото процесорно време при кооперативната схема. Броят неуспешни атомарни обновявания е 8 при повече от хиляда и шестстотин приемания, тоест състезанието за споделеното най-добро решение е практически нулево. Това е очаквано и е следствие от проектното решение: атомарната стойност се чете често и се пише само при подобрене, а подобренията стават все по-редки

с напредване на търсенето. Островната схема не отчита нито едно неуспешно атомарно обновяване, защото не използва такова, а пощенски кутии с мютекс.

Прави впечатление, че островната схема плаща повече време в синхронизация при раницата (15,0 ms) отколкото кооперативната (0,25 ms) при почти еднакъв брой приемания. Причината е в мярката: при кооперативната схема се измерва само времето вътре в критичната секция, а бързият път през атомарното четене изобщо не влиза в нея, докато при островната всяко посещение на кутията взема мютекс и се брои. Двете колони следователно не са пряко сравними и това е ограничение на инструментиранието, а не резултат.

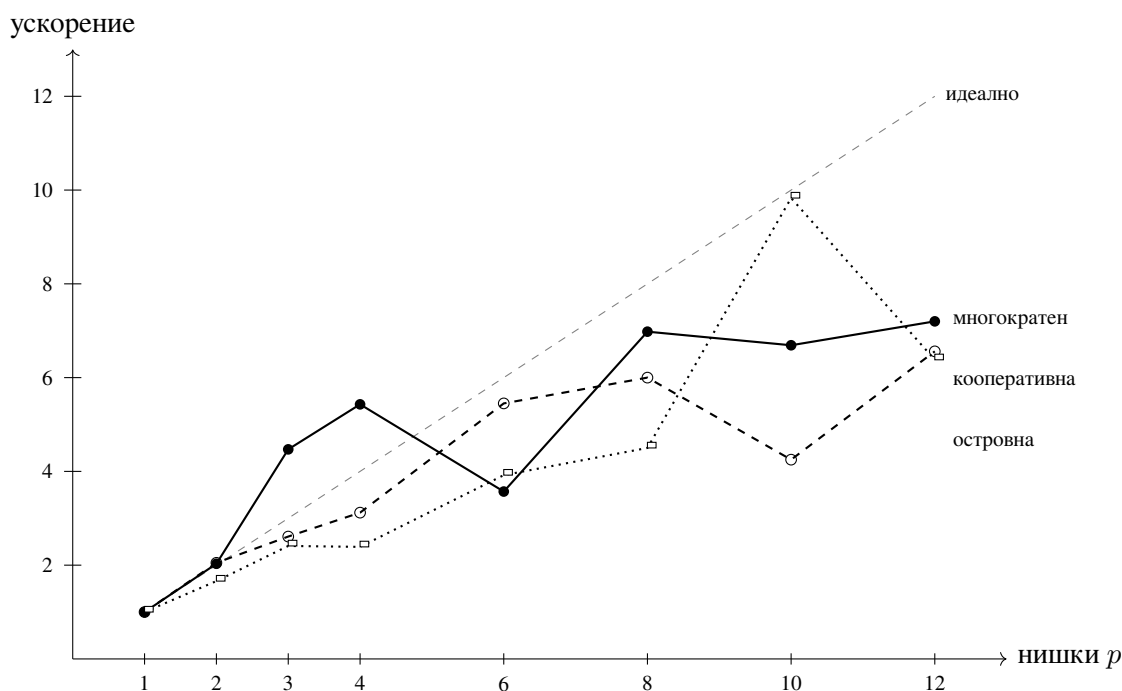
6.6. Ускорение и ефективност спрямо броя нишки

Ускорението е измерено като време до фиксиран праг. Прагът е най-лошото качество, което една нишка е постигнала в пилотна серия от пет изпълнения по 400 ms, и е 24730,9 при най-добро пилотно 24283,1. Всяка клетка е медиана от 11 изпълнения с горна граница 3000 ms (Табл. 12). Всички изпълнения достигат прага, тоест колоната с дела на успешните е 100 на сто навсякъде и не е повторена в таблицата.

Таблица 12. Време до праг 24730,9 на uniform-1000, табу търсене, 11 повторения. Команда:
anneal_bench speedup

Нишки	Многократен старт				Кооперативна			
	ms	МКР	Уск.	Ефект.	ms	МКР	Уск.	Ефект.
1	87,45	84,22	1,00	1,00	61,24	58,63	1,00	1,00
2	43,12	45,08	2,03	1,01	29,81	34,61	2,05	1,03
3	19,57	28,50	4,47	1,49	23,49	14,85	2,61	0,87
4	16,10	30,05	5,43	1,36	19,62	9,39	3,12	0,78
6	24,50	28,26	3,57	0,59	11,25	15,42	5,45	0,91
8	12,53	35,29	6,98	0,87	10,22	8,92	6,00	0,75
10	13,08	25,89	6,69	0,67	14,40	21,10	4,25	0,43
12	12,14	9,73	7,20	0,60	9,33	13,27	6,56	0,55

Островен модел				
Нишки	ms	МКР	Уск.	Ефект.
1	60,48	58,87	1,00	1,00
2	36,50	51,52	1,66	0,83
3	25,13	20,31	2,41	0,80
4	25,31	14,48	2,39	0,60
6	15,42	10,82	3,92	0,65
8	13,45	26,96	4,50	0,56
10	6,15	31,11	9,83	0,98
12	9,48	6,21	6,38	0,53



**Фигура 3. Ускорение спрямо брой нишки за трите схеми, с нанесена идеална линия.
Данните са от Табл. 12**

Кривите на Фиг. 3 се отклоняват от идеалната в две различни посоки и това е основното, което трябва да бъде обяснено (Фиг. 3).

Първо, при малък брой нишки ускорението надхвърля идеалното: 4,47 при три нишки и 5,43 при четири за независимия многократен старт, тоест ефективност съответно 1,49 и 1,36. Това не е грешка в измерването и обяснението му е в съседната колона. Междуквартилният размах при една нишка е 84,22 ms при медиана 87,45 ms, тоест разпределението на времето до праг е силно изкривено надясно. При p независими изпълнения се наблюдава минимумът от p тегления от това разпределение, а минимумът на тежкоопашатото разпределение пада по-бързо от $1/p$, докато опашката е дълга. Свръхлинейното ускорение тук следователно е свойство на разпределението на времената, а не на машината, и то изчезва, щом опашката се изчерпи: междуквартилният размах при дванадесет нишки е 9,73 ms, тоест почти цялото разсейване е погълнато.

Второ, при повече от четири нишки кривите се сплескват и ефективността пада до 0,53 до 0,60. Процесорът има шест физически ядра и дванадесет логически, тоест от седмата нишка нататък две нишки делят изпълнителните единици на едно ядро. Времето, прекарано в синхронизация, не обяснява това: то е около 1 на сто от процесорното време според Табл. 11, тоест твърде малко.

Единичните точки, които се отклоняват нагоре или надолу от тенденцията,

например 3,57 при шест нишки за многократния старт и 9,83 при десет нишки за островния модел, са в границите на разсейването: междуквартилният размах за тези клетки е съответно 28,26 и 31,11 ms при медиани 24,50 и 6,15 ms. Единадесет повторения не разграничават съседни конфигурации на това ниво на шум и заключения за отделни точки не се правят.

6.7. Фалшиво споделяне

Проектното решение състоянието на всеки работник да бъде подравнено на кеш линията беше взето предварително и подлежеше на проверка (Табл. 13). Проверката е отделен експеримент: всяка нишка увеличава собствен брояч през указател към `volatile`, така че записът да достига паметта, а разстоянието между броячите е или един, тоест осем брояча на една 64-байтова линия, или осем, тоест по един брояч на линия.

Таблица 13. Двадесет милиона увеличения на нишка. Команда: `anneal_bench falsesharing`

Нишки	Обща линия, ms	Отделни линии, ms	Отношение
2	114,3	8,4	13,53
4	267,2	24,0	11,14
8	534,8	34,9	15,35
12	519,1	41,8	12,41

Ефектът е от 11 до 15 пъти и не намалява с броя нишки. Подравняването следователно се оправдава. Уговорката е, че този експеримент измерва най-лошия възможен случай, тоест запис при всяка итерация, докато броячите в `src/parallel.cpp` се обновяват веднъж на итерация в локална за нишката структура и се записват в общия резултат едва след приключване. Измерването оправдава решението, но не твърди, че без него конкретната програма би била 13 пъти по-бавна.

6.8. Разпределение на времето в една итерация

Разпределението е получено по метода на аблацията от §5.7, с изрично записаните там ограничения (Табл. 14).

Таблица 14. Аблация на една итерация на отгряването. Команда: anneal_bench profile

Тяло на цикъла	uniform-200		uniform-1000	
	ns	дял	ns	дял
само случайно теглене	1,9	3,2 %	1,8	2,9 %
плюс генериране на ход	45,4	73,2 %	44,3	70,2 %
плюс инкрементално оценяване	50,6	8,7 %	50,5	10,3 %
плюс приемане и прилагане	59,4	14,8 %	60,6	16,6 %
пълно преоценяване, за мащаб	147,4	2×	788,8	13×

Резултатът противоречи на очакването, с което разделът за проектиране беше писан. Очакваше се оценяването на целевата функция да доминира; то заема между 8,7 и 10,3 на сто. Доминира генерирането на съседен ход, с между 70 и 73 на сто. Причината е в самия генератор: той тегли град, тегли един от шестнадесетте му най-близки съседа и прочита позицията му през обратната пермутация, тоест прави няколко непредсказуеми обръщания към паметта, докато инкременталното оценяване е четири разстояния между градове, които вече са докарани в кеша от генератора.

Последният ред дава мащаба, заради който инкременталното оценяване съществува. При 1000 града пълно преоценяване на всяка итерация би струвало 13 пъти повече от цялата итерация, тоест би направило измерванията в този раздел невъзможни за същия бюджет. При 200 града множителят е само 2, което показва, че ползата от инкременталното оценяване расте линейно с размера на инстанцията, както и се очаква.

Оттук следва и защо резултатът от §6.2 има ограничен обхват: векторизацията ускорява пълното оценяване, а пълно оценяване горещият път изобщо не прави. Тя има значение за генетичния алгоритъм и за мравчения, които оценяват цели нови решения, и почти никакво за отгряването и табу търсенето.

6.9. Обобщение на резултатите

Въпрос 1, ускорение. Времето до фиксиран праг намалява с броя нишки за трите схеми, до 7,20 пъти при дванадесет нишки за независимия многократен старт. Ускорението е свръхлинейно при две до четири нишки, което се обяснява с тежката дясна опашка на разпределението на времената, и пада до ефективност около 0,55 при дванадесет нишки,

където шестте физически ядра вече се делят от по две нишки.

Въпрос 2, кооперация. Кооперативната схема дава по-добра медиана от независимата при задачата на търговския пътник, с 0,93 на сто, и по-лоша при раницата. Цената на обмяна е измерена и е около 1 на сто от процесорното време, а състезанието за споделеното най-добро решение е практически нулево, 8 неуспешни атомарни обновявания при над 1600 приемания.

Въпрос 3, разпределение на времето. Доминира генерирането на съседен ход, с около 70 на сто. Инкременталното оценяване е под 11 на сто. Синхронизацията е около 1 на сто. Очакването, че оценяването на целевата функция доминира, е опровергано.

Въпрос 4, подредба на метаевристиките. Подредбата зависи от задачата и не се пренася между задачите. Табу търсенето води при търговския пътник и изоставя най-много при раницата; генетичният алгоритъм води при раницата; отгряването и табу търсенето решават оцветяването точно, а популационните методи не.

Въпрос 5, векторизация. Формата без разклонение на целевата функция на раницата е между 1,17 и 1,52 пъти по-бърза от очевидната, като предимството расте с размера. Дължината на маршрута не се векторизира в нито една форма, защото е сума с непряка адресация. Практическото значение на този резултат е малко, защото горещият път на двете траекторни метаевристики не прави пълно оценяване; това е отрицателен резултат и се докладва като такъв.

Неотговорени въпроси. Измерванията не установяват дали сплескването на ускорението над шест нишки се дължи на споделянето на изпълнителни единици между две логически нишки на едно ядро или на пропускателната способност на паметта. Разграничение между двете изисква хардуерни броячи, каквито не бяха снети, тоест въпросът остава отворен и не се заобикаля с правдоподобно обяснение.

ЗАКЛЮЧЕНИЕ

В рамките на проекта е проектирана и реализирана паралелна рамка за метаевристична оптимизация на комбинаторни задачи, в която задачата, метаевристиката и схемата за паралелно изпълнение са три независими точки на разширение. Реализирани са четири метаевристики и три схеми за паралелно изпълнение, а към тях е изградена методика за измерване, която фиксира входните инстанции, критерия за спиране, броя повторения и начина на обобщаване. Рамката се изгражда без нито една външна зависимост и се покрива от 28 автоматични теста.

Измерванията дават три резултата, които си заслужава да бъдат отделени. Първо, времето до фиксиран праг качество намалява до 7,20 пъти при дванадесет нишки за независимия многократен старт, като при две до четири нишки ускорението е свръхлинейно, до 5,43 при четири. Обяснението не е в машината, а в разпределението: междуквартилният размах при една нишка е 84,22 ms при медиана 87,45 ms, тоест опашката е тежка и минимумът от няколко независими тегления пада по-бързо от $1/p$. Второ, кооперативната схема подобрява медианното качество с 0,93 на сто спрямо независимата при задачата на търговския пътник и я влошава при раницата, тоест обменът не е универсално подобрение; цената му е около 1 на сто от процесорното време. Трето, подредбата на четирите метаевристики зависи от задачата и не се пренася между задачите.

Предимства и недостатъци на решението

Предимствата се формулират спрямо измереното. Разделянето на договорите работи: добавянето на трета задача не наложи промяна в нито един от четирите алгоритъма, а смяната на схемата за изпълнение не наложи промяна в нито една задача. Единната мярка за бюджет в оценявания на целевата функция позволява алгоритми, чиито итерации се различават с четири порядъка по цена, да бъдат сравнявани и да обменят решения смислено. Инкременталното оценяване се изплаща и е измерено колко: при 1000 града пълното преоценяване на всяка итерация би струвало 13 пъти повече от цялата итерация.

Недостатъците са три и всеки от тях е показан от измерване, а не от преглед на кода. Първо, качеството зависи от съседството поне толкова, колкото от избраната стратегия: едно и също табу търсене стига до 27579 или до 24965 в зависимост от това дали обхватът на хода е ограничен, тоест сравнението между метаевристики е валидно

само при еднакво добре подбрано съседство и за конкретните настройки. Второ, брояча за време в синхронизация мери различни неща при кооперативната и при островната схема, тоест двете колони не са пряко сравними; това е дефект на инструментирането, признат в §6.5. Трето, единадесет повторения не разграничават съседни конфигурации, когато междуквартилният размах е съизмерим с медианата, което личи по отделните точки в Табл. 12, отклоняващи се от тенденцията.

Насоки за по-нататъшна работа

Работата може да бъде продължена в няколко посоки. Най-близката следва от отворения въпрос в §6.9: разграничаването между споделяне на изпълнителни единици и ограничение по памет като причина за сплескването на ускорението изисква хардуерни броячи и следователно среда, в която такива могат да бъдат снети. Втората е разпределеното изпълнение върху няколко машини, което поставя въпроса за обmena при съществено по-висока цена на комуникацията, тоест променя баланса, измерен тук. Третата е автоматичната настройка на параметрите, която би премахнала едно от ограниченията, признати в §5.8, а именно че сравнението е между конкретни настройки, а не между методите изобщо. Задачно ориентираната среда за изпълнение остава разгледана, но неизмерена алтернатива по §2.2.

ЛИТЕРАТУРА

- [1] C. Blum and A. Roli, “Metaheuristics in combinatorial optimization: Overview and conceptual comparison,” *ACM Computing Surveys*, vol. 35, no. 3, pp. 268–308, 2003.
- [2] T. G. Crainic and M. Toulouse, “Parallel meta-heuristics,” in *Handbook of Metaheuristics* (M. Gendreau and J.-Y. Potvin, eds.), pp. 497–541, Boston, MA: Springer, 2 ed., 2010.
- [3] E.-G. Talbi, *Metaheuristics: From Design to Implementation*. Hoboken, NJ: John Wiley & Sons, 2009.
- [4] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, “Optimization by simulated annealing,” *Science*, vol. 220, no. 4598, pp. 671–680, 1983.
- [5] F. Glover, “Future paths for integer programming and links to artificial intelligence,” *Computers & Operations Research*, vol. 13, no. 5, pp. 533–549, 1986.
- [6] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Ann Arbor: University of Michigan Press, 1975.
- [7] M. Dorigo, V. Maniezzo, and A. Colorni, “Ant system: Optimization by a colony of cooperating agents,” *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, vol. 26, no. 1, pp. 29–41, 1996.
- [8] E. Alba, ed., *Parallel Metaheuristics: A New Class of Algorithms*. Hoboken, NJ: John Wiley & Sons, 2005.
- [9] S. Williams, A. Waterman, and D. Patterson, “Roofline: An insightful visual performance model for multicore architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [10] G. Reinelt, “TSPLIB — a traveling salesman problem library,” *ORSA Journal on Computing*, vol. 3, no. 4, pp. 376–384, 1991.
- [11] J. E. Beasley, “OR-Library: Distributing test problems by electronic mail,” *Journal of the Operational Research Society*, vol. 41, no. 11, pp. 1069–1072, 1990.
- [12] International Organization for Standardization, Geneva, *ISO/IEC 14882:2020 — Programming languages — C++*, 2020.

ПРИЛОЖЕНИЯ

Приложение А. Договори на рамката

Двата договора, върху които е построено всичко останало: Problem описва задача, Metaheuristic описва търсене. Никой от двата не знае нищо за нишки.

```
1 // The two contracts the whole framework is built around: a problem and a
2 // metaheuristic. Everything else in the library depends on these and on
3 // nothing else, which is what lets the problem, the algorithm and the
4 // execution scheme vary independently.
5 #pragma once
6
7 #include <cstddef>
8 #include <cstdint>
9 #include <limits>
10 #include <memory>
11 #include <string>
12 #include <vector>
13
14 #include "anneal/rng.hpp"
15
16 namespace anneal {
17
18 // One representation serves all three problems. For the travelling salesman
19 // problem the vector is a permutation of the cities, for the knapsack it is
20 ↪ a
21 // 0/1 indicator over the items, and for graph colouring it is a colour index
22 // per vertex. Keeping a single contiguous type means a move, a crossover and
23 ↪ a
24 // migration are the same memory operation everywhere, and it keeps the
25 // interface free of templates.
26 using Solution = std::vector<int>;
27
28 // Everything is minimised. A problem whose natural form is a maximisation
29 // negates in evaluate() and undoes the sign in display().
30 inline constexpr double kInfinity = std::numeric_limits<double>::infinity();
31
32 // A move is two indices; each problem decides what they mean. TSP: reverse
```

```

↪ the
31 // segment between positions i and j (2-opt). Knapsack: flip item i, and if
32 // j >= 0 flip item j too, which is a swap move and escapes the local optimum
33 // that single flips leave you in once the sack is full. Colouring: give
34 // vertex i the colour j.
35 struct Move {
36     int i = 0;
37     int j = 0;
38 };
39
40 // A solution together with the cached state that makes incremental
↪ evaluation
41 // and move generation cheap.
42 //
43 // 'aux' is one scalar: without it the knapsack delta would have to re-sum
↪ the
44 // weights, which is linear and would defeat the point of having a delta.
45 // 'index' is an integer array whose meaning is the problem's own: the
46 // travelling salesman problem keeps the inverse permutation there, which is
47 // what lets it propose a move by naming a city rather than a tour position.
48 // Both are maintained by apply() and rebuilt from scratch by Problem::make()
↪ .
49 struct Candidate {
50     Solution x;
51     double cost = kInfinity;
52     double aux = 0.0;
53     std::vector<int> index;
54 };
55
56 class Problem {
57 public:
58     virtual ~Problem() = default;
59
60     virtual std::string name() const = 0;
61     virtual std::size_t size() const = 0;
62
63     // Full evaluation, used to seed a Candidate and to check the deltas.
64     virtual double evaluate(const Solution& x) const = 0;
65     // Problem specific cached scalar, recomputed from scratch.

```

```

66     virtual double aux_of(const Solution&) const { return 0.0; }
67     // Problem specific cached index, recomputed from scratch. A problem that
68     // needs none leaves the vector empty, which costs nothing.
69     virtual void build_index(const Solution&, std::vector<int>& out) const {
↪ out.clear(); }
70     virtual bool feasible(const Solution& x) const = 0;
71
72     // Construction heuristic: randomised greedy, the starting point every
73     // algorithm uses unless a test asks for a purely random one.
74     virtual Solution construct(Rng& rng) const = 0;
75     virtual Solution random_solution(Rng& rng) const = 0;
76
77     // Neighbourhood.
78     virtual Move random_move(const Candidate& c, Rng& rng) const = 0;
79     virtual double delta(const Candidate& c, const Move& m) const = 0;
80     virtual void apply(Candidate& c, const Move& m, double d) const = 0;
81
82     // Attribute of a move, for the tabu list. The default treats the index
83     // pair itself as the attribute, which is the right granularity for all
84     // three problems here: it forbids undoing the exact move just made.
85     virtual std::uint64_t move_key(const Move& m) const {
86         return static_cast<std::uint64_t>(m.i) * 1000003ULL +
87             static_cast<std::uint64_t>(m.j + 1);
88     }
89
90     // Recombination for the genetic algorithm. Each problem owns this
↪ because
91     // a uniform crossover would destroy a permutation, and an order
↪ crossover
92     // is meaningless for a bit vector.
93     virtual Solution crossover(const Solution& a, const Solution& b, Rng& rng
↪ ) const = 0;
94
95     // Ant colony support. The algorithm owns the pheromone dynamics, meaning
96     // evaporation, deposit amount and the choice of which ant deposits; the
97     // problem owns what a pheromone entry means and how a trail is walked.
98     virtual std::size_t pheromone_size() const = 0;
99     virtual Solution construct_aco(const std::vector<double>& tau, double
↪ alpha,

```

```

100         double beta, Rng& rng) const = 0;
101     virtual void deposit(const Solution& x, double amount,
102         std::vector<double>& tau) const = 0;
103
104     // Known optimum in internal (minimised) units, or NaN when unknown.
105     virtual double best_known() const { return std::numeric_limits<double>::
↪ quiet_NaN(); }
106     // Internal cost converted back to the problem's natural units.
107     virtual double display(double cost) const { return cost; }
108     virtual const char* unit() const { return "cost"; }
109
110     Candidate make(const Solution& x) const {
111         Candidate c;
112         c.x = x;
113         c.cost = evaluate(c.x);
114         c.aux = aux_of(c.x);
115         build_index(c.x, c.index);
116         return c;
117     }
118 };
119
120 // A metaheuristic is an object with state that takes steps, not a function
121 // that returns an answer. The parallel schemes need to interrupt a search,
122 // read its best solution and hand it a better one from elsewhere, and none
↪ of
123 // that is expressible if the search is a single call that runs to completion
↪ .
124 class Metaheuristic {
125 public:
126     virtual ~Metaheuristic() = default;
127
128     virtual std::string name() const = 0;
129     // One iteration. What an iteration is differs per algorithm: one move
↪ for
130     // annealing, one generation for the genetic algorithm, one colony sweep
131     // for ant colony optimisation. That is exactly why the budget is counted
132     // in objective evaluations and wall clock, never in steps.
133     virtual void step() = 0;
134     virtual const Solution& best() const = 0;

```

```

135     virtual double best_cost() const = 0;
136     // Accept a solution found elsewhere. Returns true if it was adopted.
137     virtual bool inject(const Solution& x, double cost) = 0;
138     virtual std::uint64_t evaluations() const = 0;
139 };
140
141 using MetaheuristicPtr = std::unique_ptr<Metaheuristic>;
142
143 } // namespace anneal

```

Листинг 1: include/anneal/core.hpp

Приложение Б. Споделено най-добро решение

Единственото споделено изменяемо състояние в кооперативната схема. Атомарната цена е бързият път, а решението се записва под мютекс с повторна проверка вътре в критичната секция; мотивът е обяснен в §4.6.

```

54 bool SharedBest::offer(const Solution& x, double cost, WorkerMetrics& metrics
    ↪ ) {
55     double current = cost_.load(std::memory_order_acquire);
56     if (!(cost < current)) return false;
57     const Clock::time_point t0 = Clock::now();
58     bool won = false;
59     while (cost < current) {
60         if (cost_.compare_exchange_weak(current, cost, std::
    ↪ memory_order_acq_rel,
61                                     std::memory_order_acquire)) {
62             won = true;
63             break;
64         }
65         // compare_exchange_weak refreshes 'current' on failure and may also
66         // fail spuriously. Both are counted: the number is reported as
67         // contention on the shared best, and it is the honest measure of it.
68         ++metrics.cas_failures;
69     }
70     if (won) {
71         std::lock_guard<std::mutex> guard(mu_);

```

```

72         if (cost <= stored_cost_) {
73             best_ = x;
74             stored_cost_ = cost;
75             version_.fetch_add(1, std::memory_order_release);
76         }
77     }
78     metrics.sync_wait_ns += static_cast<std::uint64_t>(
79         std::chrono::duration_cast<std::chrono::nanoseconds>(Clock::now() -
↪ t0).count());
80     return won;
81 }
82
83 bool SharedBest::read(Solution& out, double& cost, std::uint64_t&
↪ seen_version,
84                     WorkerMetrics& metrics) const {
85     const std::uint64_t version = version_.load(std::memory_order_acquire);
86     if (version == seen_version) return false;
87     const Clock::time_point t0 = Clock::now();
88     {
89         std::lock_guard<std::mutex> guard(mu_);
90         out = best_;
91         cost = stored_cost_;
92         seen_version = version_.load(std::memory_order_relaxed);
93     }
94     metrics.sync_wait_ns += static_cast<std::uint64_t>(
95         std::chrono::duration_cast<std::chrono::nanoseconds>(Clock::now() -
↪ t0).count());
96     return !out.empty();
97 }
98
99 double SharedBest::cost() const {
100     std::lock_guard<std::mutex> guard(mu_);
101     return stored_cost_;
102 }
103
104 Solution SharedBest::snapshot() const {
105     std::lock_guard<std::mutex> guard(mu_);

```

Листинг 2: src/parallel.cpp, SharedBest::offer и SharedBest::read

Приложение В. Обща част на трите схеми

Трите схеми се различават само по обратното извикване `exchange`. Всичко останало, тоест създаването на работниците, спирането по бюджет и събирането на резултата, е общо и е написано веднъж.

```
137 template <typename Exchange>
138 RunResult run_workers(const Factory& factory, int threads, const Budget&
    ↪ budget,
139                      std::uint64_t seed, Exchange exchange) {
140     if (threads < 1) threads = 1;
141     const std::size_t count = static_cast<std::size_t>(threads);
142     const std::uint64_t per_worker_evaluations =
143         budget.evaluations == 0 ? 0
144           : std::max<std::uint64_t>(1, budget.
    ↪ evaluations / count);
145
146     std::vector<Worker> workers(count);
147     std::atomic<bool> stop{false};
148     std::atomic<long long> target_ns{-1};
149     const Clock::time_point started = Clock::now();
150
151     {
152         std::vector<std::jthread> pool;
153         pool.reserve(count);
154         for (std::size_t w = 0; w < count; ++w) {
155             pool.emplace_back([&, w] {
156                 Worker& self = workers[w];
157                 self.search = factory(Rng::derive(seed, w), static_cast<int>(
    ↪ w));
158
159                 self.best = self.search->best();
160                 self.best_cost = self.search->best_cost();
161                 Deadline deadline(budget.wall);
162                 for (;;) {
163                     if (stop.load(std::memory_order_relaxed)) break;
164                     if (deadline.expired()) break;
165                     if (per_worker_evaluations != 0 &&
                        self.search->evaluations() >= per_worker_evaluations)
```



```

166         break;
167     }
168     self.search->step();
169     ++self.metrics.iterations;
170     if (self.search->best_cost() < self.best_cost) {
171         self.best_cost = self.search->best_cost();
172         self.best = self.search->best();
173         ++self.metrics.improvements;
174     }
175     exchange(self, static_cast<int>(w));
176     if (self.best_cost <= budget.target) {
177         long long expected = -1;
178         const long long elapsed =
179             std::chrono::duration_cast<std::chrono::
180                 ↪ nanoseconds>(
181                 Clock::now() - started)
182                 .count();
183         target_ns.compare_exchange_strong(expected, elapsed);
184         stop.store(true, std::memory_order_relaxed);
185         break;
186     }
187     self.metrics.evaluations = self.search->evaluations();
188     });
189 }
190 // std::jthread joins on destruction, so an exception thrown anywhere
191 // above cannot leave a worker running behind the returning frame.
192 }
193
194 RunResult result;
195 result.wall_ms = ms_since(started);
196 result.workers.reserve(count);
197 for (Worker& w : workers) {
198     result.workers.push_back(w.metrics);
199     result.evaluations += w.metrics.evaluations;
200     if (w.best_cost < result.cost) {
201         result.cost = w.best_cost;
202         result.best = w.best;

```

```

203     }
204 }
205 const long long ns = target_ns.load();
206 if (ns >= 0) {
207     result.reached_target = true;
208     result.time_to_target_ms = static_cast<double>(ns) / 1e6;
209 }
210 return result;
211 }
212
213 } // namespace

```

Листинг 3: src/parallel.cpp, run_workers

Приложение Г. Островен модел с миграция по пръстен

```

260 RunResult run_island(const Factory& factory, int threads, const Budget&
↳ budget,
261                     std::uint64_t seed, std::uint64_t migration_evaluations)
↳ {
262     if (threads < 1) threads = 1;
263     const std::size_t count = static_cast<std::size_t>(threads);
264     // One mailbox per island. unique_ptr because a mutex is neither copyable
265     // nor movable, so the vector cannot hold them by value and still grow.
266     std::vector<std::unique_ptr<Mailbox>> boxes;
267     boxes.reserve(count);
268     for (std::size_t i = 0; i < count; ++i) boxes.push_back(std::make_unique<
↳ Mailbox>());
269     const std::uint64_t interval = std::max<std::uint64_t>(1,
↳ migration_evaluations);
270
271     return run_workers(factory, threads, budget, seed, [&](Worker& self, int
↳ id) {
272         if (!due_to_exchange(self, interval)) return;
273         const Clock::time_point t0 = Clock::now();
274         // Ring topology: island i emigrates to island i+1 only. A fully
275         // connected topology spreads a good solution in one hop and
↳ collapses

```

```

276 // into the cooperative scheme; the ring keeps the islands genuinely
277 // separate for a number of migrations proportional to their count.
278 const std::size_t next = (static_cast<std::size_t>(id) + 1) % count;
279 {
280     Mailbox& out = *boxes[next];
281     std::lock_guard<std::mutex> guard(out.mu);
282     if (self.best_cost < out.cost) {
283         out.solution = self.best;
284         out.cost = self.best_cost;
285         out.full = true;
286         ++self.metrics.offers;
287     }
288 }
289 Solution incoming;
290 double incoming_cost = kInfinity;
291 {
292     Mailbox& in = *boxes[static_cast<std::size_t>(id)];
293     std::lock_guard<std::mutex> guard(in.mu);
294     if (in.full) {
295         incoming = in.solution;
296         incoming_cost = in.cost;
297         in.full = false;
298     }
299 }
300 self.metrics.sync_wait_ns += static_cast<std::uint64_t>(
301     std::chrono::duration_cast<std::chrono::nanoseconds>(Clock::now()
↪ - t0).count());
302 if (!incoming.empty() && self.search->inject(incoming, incoming_cost)
↪ ) {
303     ++self.metrics.adopted;
304     if (incoming_cost < self.best_cost) {
305         self.best_cost = incoming_cost;
306         self.best = incoming;
307     }
308 }
309 });
310 }

```

Листинг 4: src/parallel.cpp, run_island

Приложение Д. Генератор на съседен ход при търговския пътник

Функцията с най-голямо измерено влияние върху качеството, заедно с коментара, който записва защо първата ѝ версия беше грешна.

```
160 Move TspProblem::random_move(const Candidate& c, Rng& rng) const {
161     // 2-opt: reverse the tour segment between two positions. How those two
162     // positions are chosen is the single most consequential decision in this
163     // file, so both forms are here and the weaker one is the fallback.
164     const std::size_t n = c.x.size();
165     Move m;
166
167     // Preferred form: name a city, pick one of its sixteen nearest
168     // neighbours, and let the inverse permutation say where that neighbour
169     // sits in the tour. The move then always installs a short edge.
170     //
171     // Measured effect, from anneal_bench neighbourhood: it raises the share
172     // of proposals that improve the tour from 0.17 per cent to 3.3 per cent,
173     // a factor of twenty, but at two million proposals the descent ends in
174     // much the same place either way, 24965 against 25087. Its real value is
175     // elsewhere. It shrinks the distribution of uphill moves, whose median
176     // falls from 355 to 82, and simulated annealing calibrates its initial
177     // temperature from exactly that distribution, so with uniform sampling
178     // the chain starts four times too hot and never beats its own
179     // construction inside the budget.
180     if (candidate_moves_ && candidates_ > 0 && c.index.size() == n_) {
181         const std::size_t from = static_cast<std::size_t>(rng.below(n));
182         const std::size_t city = static_cast<std::size_t>(c.x[from]);
183         const std::size_t pick = static_cast<std::size_t>(rng.below(
184     ↪ candidates_));
185         const std::size_t neighbour = static_cast<std::size_t>(near_[city *
186     ↪ candidates_ + pick]);
187         const std::size_t to = static_cast<std::size_t>(c.index[neighbour]);
188         std::size_t lo = std::min(from, to);
189         std::size_t hi = std::max(from, to);
190         if (lo != hi && hi - lo <= span_ && !(lo == 0 && hi == n - 1)) {
191             m.i = static_cast<int>(lo);
192             m.j = static_cast<int>(hi);
193             return m;
194         }
195     }
```

```

192     }
193     // The neighbour is too far away along the tour to reverse cheaply,
↪ or
194     // is already adjacent. Falling through to the bounded random move
195     // keeps the neighbourhood connected, so no tour is unreachable.
196 }
197
198 m.i = static_cast<int>(rng.below(n - 1));
199 const std::size_t remaining = n - 1 - static_cast<std::size_t>(m.i);
200 const std::size_t reach = std::min(span_, remaining);
201 m.j = m.i + 1 + static_cast<int>(rng.below(reach));
202 if (m.i == 0 && m.j == static_cast<int>(n) - 1) m.j = static_cast<int>(n)
↪ - 2;
203 return m;
204 }
205
206 double TspProblem::delta(const Candidate& c, const Move& m) const {
207     const Solution& t = c.x;
208     const std::size_t n = t.size();
209     const std::size_t i = static_cast<std::size_t>(m.i);
210     const std::size_t j = static_cast<std::size_t>(m.j);
211     if (i >= j || j >= n) return 0.0;
212     const std::size_t jn = (j + 1) % n;
213     if (jn == i) return 0.0;
214     const int a = t[i];
215     const int b = t[i + 1];
216     const int c2 = t[j];
217     const int d2 = t[jn];
218     // Removing edges (a,b) and (c2,d2), adding (a,c2) and (b,d2). Four table
219     // lookups regardless of the segment length, against O(n) for a full
220     // re-evaluation.
221     return distance(a, c2) + distance(b, d2) - distance(a, b) - distance(c2,
↪ d2);
222 }
223
224 void TspProblem::apply(Candidate& c, const Move& m, double d) const {
225     const auto first = c.x.begin() + m.i + 1;
226     const auto last = c.x.begin() + m.j + 1;
227     std::reverse(first, last);

```

```

228 // The inverse permutation only changes inside the reversed segment, so
229 // maintaining it costs the same order as the reversal itself.
230 if (c.index.size() == n_) {
231     for (int position = m.i + 1; position <= m.j; ++position) {
232         c.index[static_cast<std::size_t>(c.x[static_cast<std::size_t>(
↪ position))]] = position;
233     }
234 }
235 c.cost += d;
236 }

```

Листинг 5: src/tsp.cpp, random_move и delta

Приложение Е. Използвани еталонни инстанции

Всички инстанции се генерират от програмата. Произходът на обявената стойност за всяка от тях е даден в Табл. 6. Реализиран е и четец на формата TSPLIB [10] за подмножеството EUC_2D, за да може публикувана инстанция да бъде подадена отвън, но нито едно измерване в Раздел VI не зависи от изтеглен файл.

Приложение Ж. Команди за възпроизвеждане на измерванията

Всички команди са изпълнявани от главната директория на хранилището. Версиите на инструментите са в Табл. 5.

```

1 cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release
2 cmake --build build
3 ctest --test-dir build --output-on-failure
4 cmake --build build --target demo

```

Листинг 6: Изграждане, тестове, демонстрация

```

1 ./build/anneal_bench objective      # Табл. 6.1 и 6.2
2 ./build/anneal_bench neighbourhood # Табл. 6.3
3 ./build/anneal_bench quality        # Табл. 6.4
4 ./build/anneal_bench schemes        # Табл. 6.5
5 ./build/anneal_bench speedup        # Табл. 6.6 и Фиг. 6.1
6 ./build/anneal_bench falsesharing   # Табл. 6.7
7 ./build/anneal_bench profile        # Табл. 6.8
8 ./build/anneal_bench all            # всички наведнъж

```

Приложение 3. Изход на демонстрационната програма

Изходът по-долу е дословен от изпълнение на машината от Табл. 5. Демонстрацията избира сама най-добрия по медиана алгоритъм в част 1 и го използва в останалите части, тоест изборът не е фиксиран в текста.

```
-----
Anneal: parallel metaheuristics, measured
-----

compiler      : g++ 15.2.0
logical CPUs   : 12
clock         : std::chrono::steady_clock
instance      : uniform-1000, 1000 cities, uniform in a square
optimum       : not known for this instance, so quality is a tour length
-----

Part 1. Four metaheuristics, 1 thread, 250 ms budget, 7 seeds
-----

algorithm      best      median      worst      IQR      median evals
-----
annealing      25177.4    25417.2    25517.0     97.5      1810831
tabu           24329.4    24452.6    24734.7    146.0      4085681
genetic        26665.2    27263.9    27913.3    552.3       50304
aco            27128.2    27251.9    27328.3     92.4       1217
-----

Lower is better. Best, median and worst are over the 7 seeds, and the
interquartile range is the spread a single run would have hidden.
Best median quality: tabu.
-----

Part 2. Speedup and efficiency of tabu, time to a fixed target
-----

pilot, 1 thread, 400 ms, 5 seeds : best 24232.1, worst 24580.3
target tour length                : 24580.3
cap per repetition                 : 3000 ms, 7 seeds per thread count
-----

threads    median t2t ms    IQR ms    speedup    efficiency    hits
-----
1           127.0           164.3      1.00      1.00      100%
2           129.4           54.3       0.98      0.49      100%
3           100.5           36.9       1.26      0.42      100%
4            65.5           57.5       1.94      0.48      100%
6            71.5           40.7       1.78      0.30      100%
8            85.0           57.7       1.49      0.19      100%
10           91.1           85.4       1.39      0.14      100%
12           58.8           83.0       2.16      0.18      100%
-----

Speedup is the median time of one thread divided by the median time of p
threads. Efficiency is that speedup divided by p. Both come from medians of
seven runs, and hits is the share of runs that reached the target at all.
-----

Part 3. Three parallel schemes, tabu, 8 threads, 300 ms budget
-----
```

scheme	best	median	worst	sync ms total	CAS retries	adopted
multistart	24198.3	24316.3	24409.3	0.000	0	0
cooperative	24072.3	24187.2	24341.0	11.147	7	1784
island	23929.2	23965.1	24305.2	8.563	0	964

The sync column is the time all workers together spent inside the shared best or a mailbox, against 300 ms of budget per worker.

Part 4. Instances with a known optimum, 8 threads, 300 ms budget, cooperative

instance	unit	known value	median found	median gap
grid-24x24	tour length	5760.00	5760.00	0.00 %
knapsack-400	packed value	15771.00	15591.00	1.14 %
coloring-150-k8	conflicting edges	0.00	0.00	0 absolute

The `grid` optimum is exact: no tour edge can be shorter than the spacing, and a tour attaining that bound exists for an even side. The knapsack optimum is exact by dynamic programming over the generated instance. The colouring optimum is zero conflicts, reachable because the instance is generated from a planted proper colouring, so its gap is reported as an absolute count.

Листинг 8: ./build/anneal_demo

Приложение И. Изпълнител на тестовете

Изпълнителят е деветдесет реда и няма външна зависимост. Това е съзнателно: рамка за тестове, изтегляна при конфигуриране, е зависимост, която трето лице трябва да удовлетвори, преди първото изграждане изобщо да проработи.

```

1 // Assert based test runner. No external framework: the project has to build
  ↳ on
2 // a machine with a compiler and CMake and nothing else, and a test framework
3 // fetched at configure time is a dependency the reader has to satisfy before
4 // the first build works.
5 #pragma once
6
7 #include <cmath>
8 #include <cstdio>
9 #include <exception>
10 #include <functional>
11 #include <string>
12 #include <utility>
13 #include <vector>
14
15 namespace check {

```



```

16
17 struct Case {
18     std::string name;
19     std::function<void()> body;
20 };
21
22 inline std::vector<Case>& registry() {
23     static std::vector<Case> cases;
24     return cases;
25 }
26
27 struct Registrar {
28     Registrar(const char* name, std::function<void()> body) {
29         registry().push_back(Case{name, std::move(body)});
30     }
31 };
32
33 struct Failure {
34     std::string what;
35 };
36
37 inline void fail(const char* file, int line, const std::string& what) {
38     char buf[64];
39     std::snprintf(buf, sizeof buf, ":%d: ", line);
40     throw Failure{std::string(file) + buf + what};
41 }
42
43 inline int run(int argc, char** argv) {
44     const std::string filter = argc > 1 ? argv[1] : std::string();
45     int ran = 0;
46     int failed = 0;
47     for (const Case& c : registry()) {
48         if (!filter.empty() && c.name != filter) continue;
49         ++ran;
50         try {
51             c.body();
52             std::printf("[ ok ] %s\n", c.name.c_str());
53         } catch (const Failure& f) {
54             std::printf("[FAIL] %s\n      %s\n", c.name.c_str(), f.what.

```

```

↪ c_str());
55         ++failed;
56     } catch (const std::exception& e) {
57         std::printf("[FAIL] %s\n      unexpected exception: %s\n", c.
↪ name.c_str(),
58                     e.what());
59         ++failed;
60     }
61 }
62 if (ran == 0) {
63     std::printf("no test matched '%s'; the CMake list and the sources
↪ disagree\n",
64               filter.c_str());
65     return 2;
66 }
67 std::printf("%d run, %d failed\n", ran, failed);
68 return failed == 0 ? 0 : 1;
69 }
70
71 } // namespace check
72
73 #define TEST(name) \
74     static void name(); \
75     static ::check::Registrar registrar_#name(#name, name); \
76     static void name()
77
78 #define CHECK(cond)
↪ \
79     do {
↪ \
80         if (!(cond)) ::check::fail(__FILE__, __LINE__, "CHECK(" #cond ")
↪ failed"); \
81     } while (0)
82
83 #define CHECK_MSG(cond, msg)
↪ \
84     do {
↪ \
85         if (!(cond))

```

```

↪      \
86      ::check::fail(__FILE__, __LINE__,
↪      \
87      std::string("CHECK(" #cond ") failed: ") + (msg));
↪      \
88  } while (0)
89
90 #define CHECK_NEAR(a, b, eps)
↪      \
91  do {
↪      \
92      const double check_a = (a);
↪      \
93      const double check_b = (b);
↪      \
94      const double check_eps = (eps);
↪      \
95      if (!(std::fabs(check_a - check_b) <= check_eps)) {
↪      \
96          char check_buf[192];
↪      \
97          std::snprintf(check_buf, sizeof check_buf,
↪      \
98          #a " = %.12g differs from " #b " = %.12g by more
↪  than %.3g", \
99          check_a, check_b, check_eps);
↪      \
100      ::check::fail(__FILE__, __LINE__, check_buf);
↪      \
101  }
↪      \
102  } while (0)
103
104 #define CHECK_EQ(a, b)
↪      \
105  do {
↪      \
106      if (!((a) == (b)))
↪      \

```

```
107         ::check::fail(__FILE__, __LINE__, "CHECK_EQ(" #a " , " #b ")  
↪ failed");    \  
108     } while (0)
```

Листинг 9: tests/check.hpp